PHYSICS OF LANGUAGE MODELS

Part 1: Learning Hierarchical Language Structures

How Transformers Learn to Parse by Implementing Dynamic Programming

Zeyuan Allen-Zhu · Yuanzhi Li

Meta / FAIR Labs · Mohamed bin Zayed University of AI

ICML 2024

Can Language Models Understand Structure?

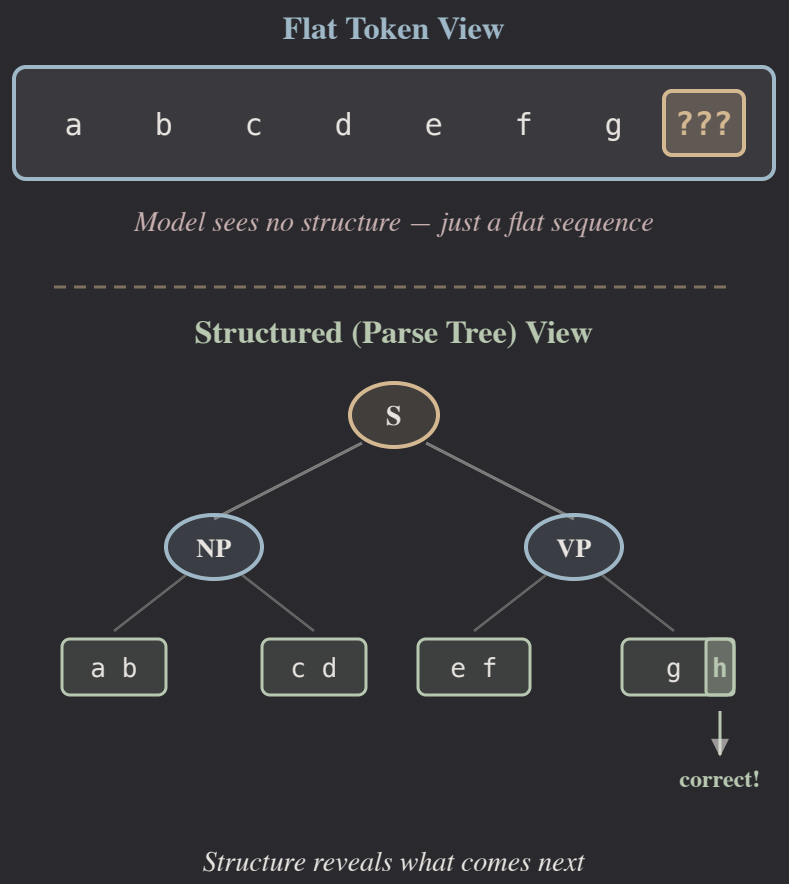
→LLMs predict the next token one step at a time

→But natural language has **deep hierarchical structure** — nested phrases, clauses, dependencies

→Does next-token prediction learn this structure, or just surface-level statistics?

→If a model sees only flat token sequences, can it recover the **hidden tree** that generated them?

> **Central Question:** Do transformers learn to represent and use hierarchical parse structure internally?



The Challenge: Next-Token Prediction Requires Global Planning

→In a structured language, the correct next token depends on **long-range context**

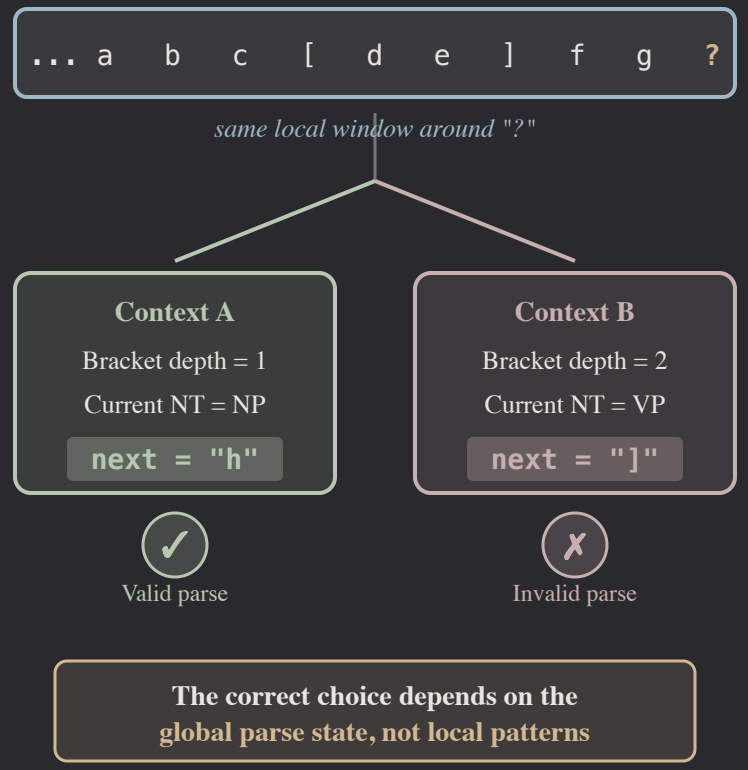
→Two positions with identical local context can require **different continuations**

→The model must track which brackets are open, how deeply nested it is, and which rules apply

→This is **not just pattern matching** — it requires maintaining a global parse state

> **Key Difficulty:** A sequence like `...Y(...)[[...]{...}]` requires knowing the full bracket stack to pick the next token correctly.

Local Context is Identical — But Correct Choice Differs



01

Background

Context-free grammars, transformers, and the gap in our understanding

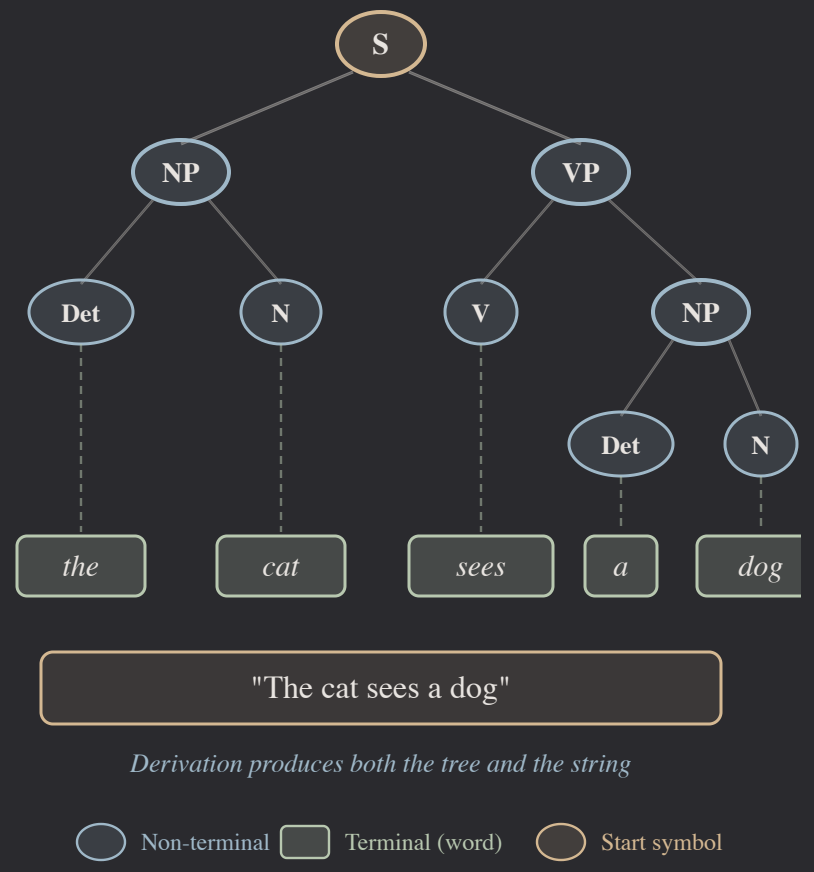
Context-Free Grammars: A Primer

A **context-free grammar** (CFG) defines a language through recursive rewriting rules:

```
S → NP VP
NP → Det N
VP → V NP
Det → "the" | "a"
N → "cat" | "dog"
V → "sees"
```

Each rule replaces one **non-terminal** with a sequence of symbols, producing **hierarchical structure**.

> **Key property:** The derivation creates a tree. Every sentence has an underlying parse tree that determines its structure.



Hierarchical Structure in Natural Language

→Real sentences have **deeply nested** phrase structure

→The **Penn TreeBank** annotates English sentences with parse trees using ~28 non-terminal categories (NP, VP, PP, ...)

→Even short sentences have **4-5 levels** of hierarchy; longer sentences can reach 8+

→Hierarchical depth grows with sentence complexity — not just length

> **Challenge:** Can a model trained on flat token sequences reconstruct these hidden trees internally?

Short sentence (5 levels deep)

Penn TreeBank parse tree (short sentence)

Longer sentence (7 levels deep)

Penn TreeBank parse tree (longer sentence)

Penn TreeBank parse trees showing NP, VP, PP, S, SBAR hierarchies

The Transformer Architecture

→GPT-2 is an **autoregressive** transformer: it predicts one token at a time, left to right

→At each step, it reads **all previous tokens** via multi-head self-attention

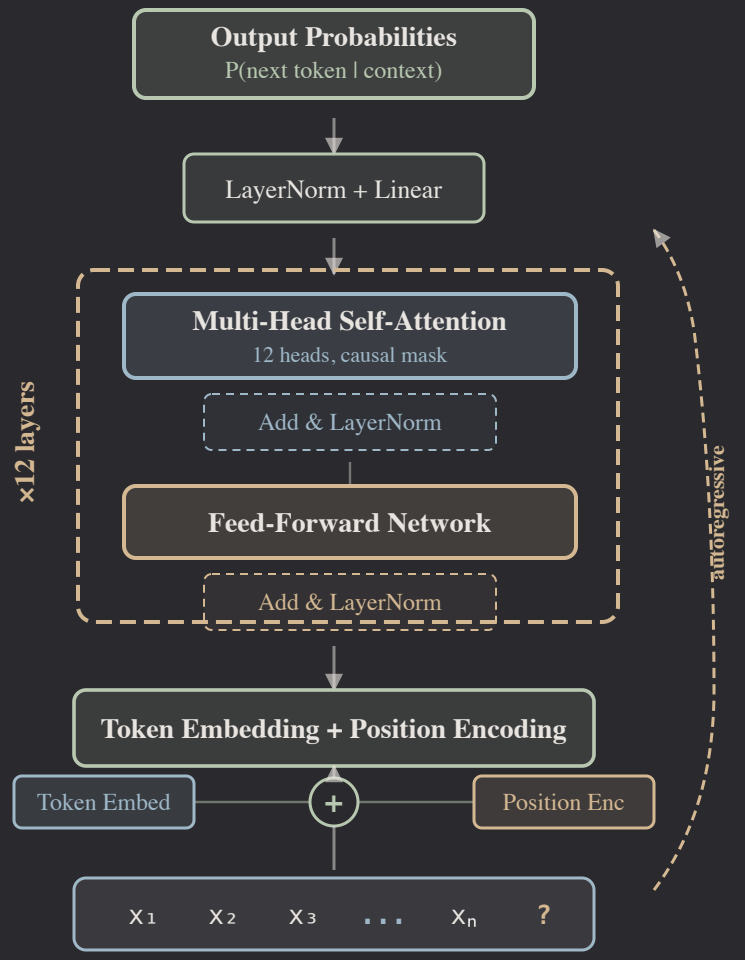
→Attention + feedforward layers are stacked into **12 layers**, building increasingly abstract representations

→A **causal mask** prevents the model from seeing future tokens

→**Position encodings** tell the model where each token sits in the sequence

> **Key question:** What kind of internal representations does this architecture learn when trained on structured (CFG) data?

This paper: GPT-2 small — 12 layers, 12 heads, 768-dim hidden, ~85M params. Trained from scratch on synthetic CFG data.



GPT-2 Small: 12 layers × 12 heads × 768 dim

Position Encodings: Absolute, Relative, Rotary

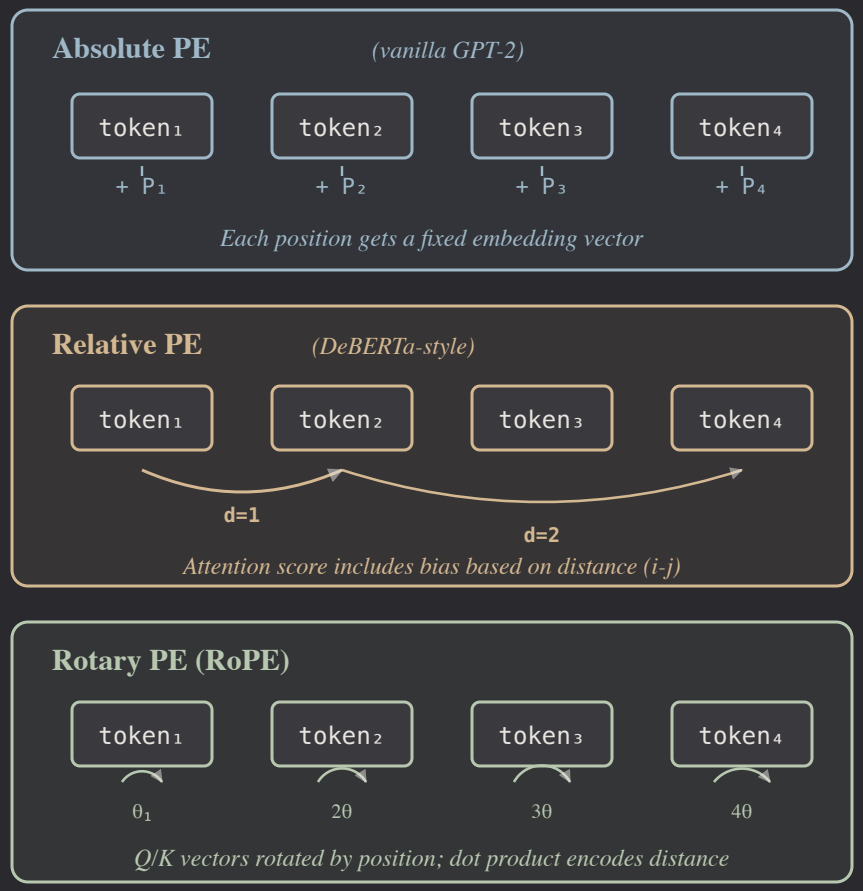
→ **Absolute PE** (vanilla GPT-2): adds a fixed vector for each position (1, 2, 3, ...)

→ **Relative PE** (DeBERTa-style): attention score depends on the *distance* between tokens, not their absolute positions

→ **Rotary PE** (RoPE): rotates query/key vectors by position angle — encodes relative distance in dot products

→ Also tested: **DeBERTa encoder** (bidirectional + relative PE) as a baseline

> **Spoiler:** Position encoding type turns out to be the single most important architectural choice for learning hierarchical structure. Relative and rotary PE succeed; absolute PE fails on deep CFGs.



Absolute: ✗ **Relative: ✓** **Rotary: ✓**

Prior Work: Formal Languages Meet Neural Networks

Several lines of work have explored what neural networks learn about language structure. Each provides important insights, but **none offers a complete mechanistic picture**.

Work	Approach	Finding	Limitation
Deletang et al. 2023	Tested NNs on Chomsky hierarchy	Transformers struggle with context-free tasks	*Very short sequences (length ≤10); toy-scale models*
Hewitt & Manning 2019	Structural probes on BERT	Parse tree distances encoded in embeddings	*Probing only — no mechanism for how structure is computed*
Olsson et al. 2022	Mechanistic interp. of induction heads	In-context copying circuits	*Simple pattern matching; not hierarchical composition*
Zhao et al. 2023	Transformers parse while predicting	Concurrent finding: implicit parsing	*Theoretical focus; limited empirical depth*
Murty et al. 2023	Intrinsic compositionality	Tree structure in representations	*Real language only; no controlled experiments*

The Gap: No prior work provides a **controlled study** showing that transformers learn hierarchical parsing at realistic scale, with both *probing evidence* (what is learned) and *mechanistic evidence* (how it is computed via attention).

The Core Discovery

Transformers don't just memorize patterns — they learn to parse

Transformers Learn to Parse by Implementing Dynamic Programming

> **Core Discovery:** GPT-2 trained on CFG data builds internal parse trees and uses **DP-like attention** to track hierarchical structure at every level.

→ **Top:** The CFG generates strings with hidden structure (NT ancestors s_3-s_6, terminals x)

→ **Bottom:** The transformer learns to encode NT boundaries and ancestors in its hidden states

→ Attention patterns concentrate at **NT boundary positions**, mimicking CKY parsing

→ Color-coded numbers show boundary counts (b♯) and position counters (p_3-p_6)

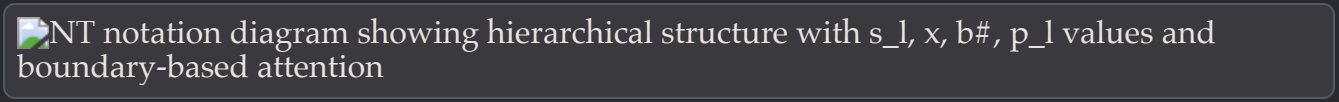


Figure from Allen-Zhu & Li (2024): NT ancestor encoding and boundary-based attention patterns

03

The Synthetic CFG Testbed

A controlled environment to study hierarchical learning in transformers

Designing Multi-Level Context-Free Grammars

→Family of synthetic CFGs with **up to 7 hierarchy levels**

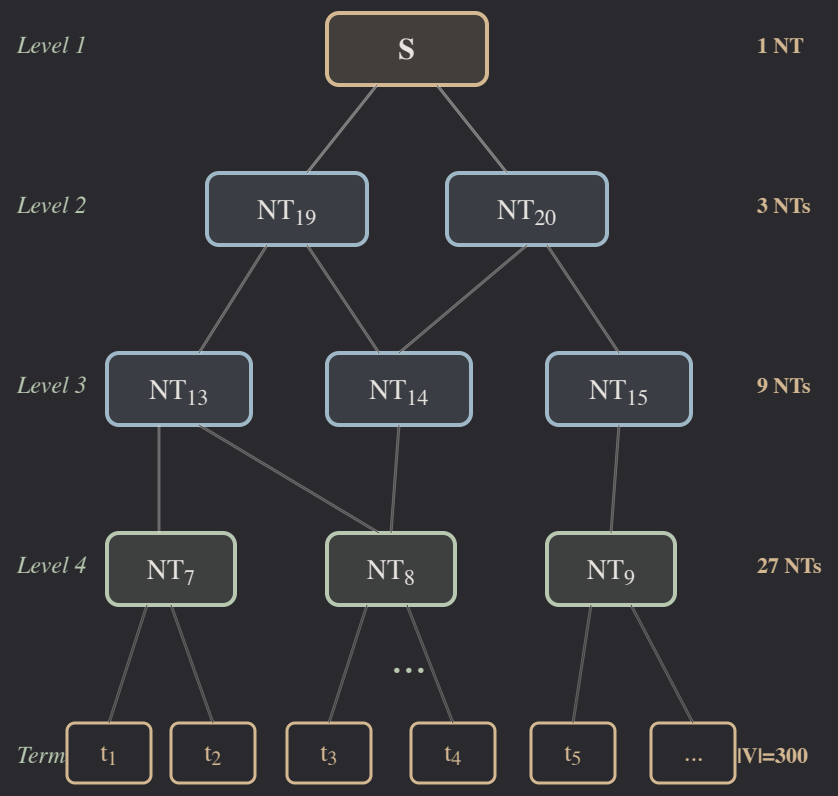
→Each non-terminal (NT) has **2-3 production rules**, each producing 2-3 children

→Sequence lengths reach **~300 tokens** with vocabulary $|V| = 300$

→Variants cfg3b through cfg3i control rule overlap, depth, and ambiguity

→NT counts per level: $(1, 3, 9, 27, 81, 27, 9)$ — deep hierarchical nesting

> **Key design goal:** Long-range dependencies + local ambiguity — next-token prediction requires *global* planning, not just local pattern matching.



Simplified CFG hierarchy (cfg3b family)

Notation: NT Ancestors, Boundaries, and Positions

$s_\ell(i)$ NT ancestor index at level ℓ for token i — which non-terminal at level ℓ generated this token

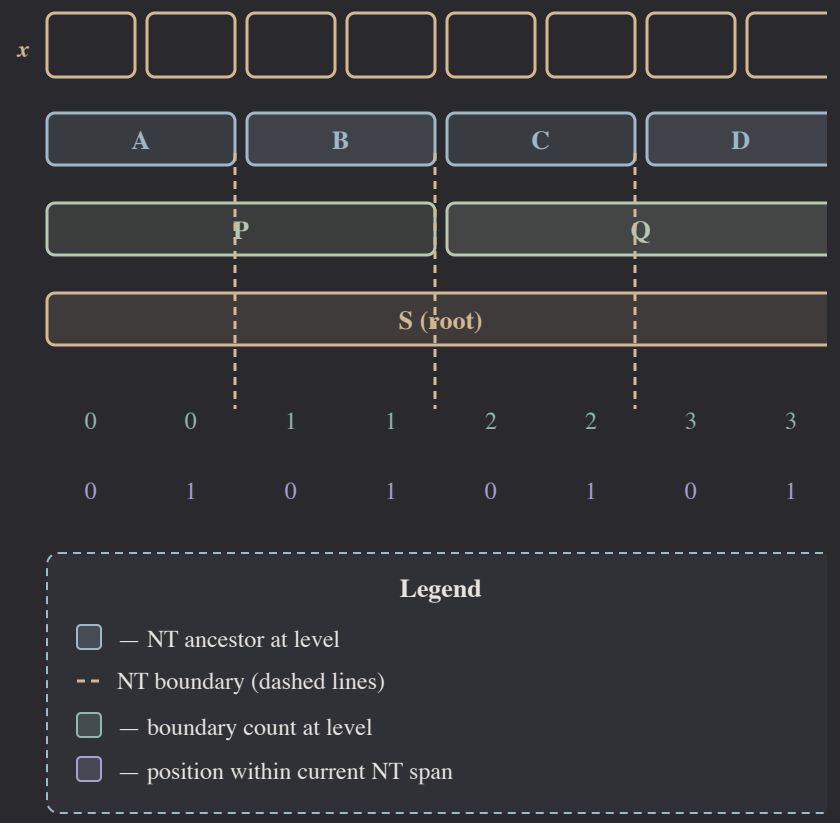
x The terminal sequence (observed tokens)

$b_\ell^{\#}(i)$ NT boundary count at level ℓ — how many level-ℓ NT boundaries have occurred before position i

$p_\ell(i)$ Position counter within the current NT at level ℓ — local offset from the start of the NT span

An **NT boundary** (or NT end) at level ℓ is a position where the derivation at level ℓ completes a subtree. The **deepest NT-end** is the finest boundary.

Example: 8-token sequence with 3 NT levels



Synthetic vs Real-Language Parse Trees

→Synthetic CFGs mirror the **hierarchical depth** of real English parse trees

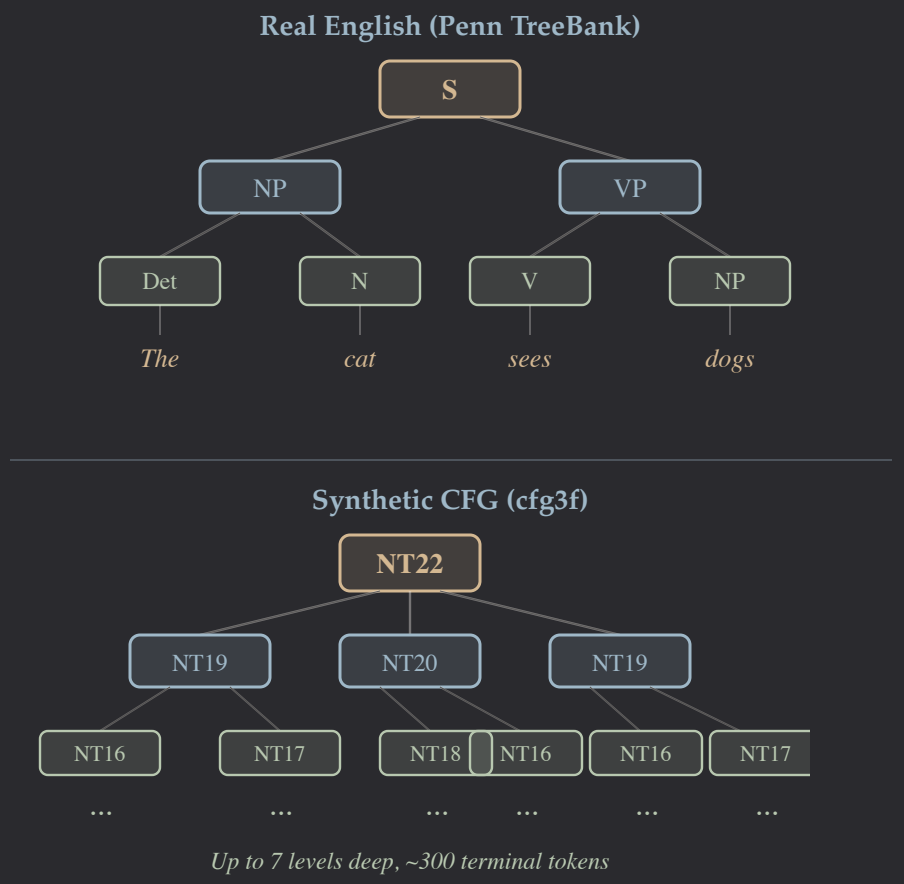
→Real-language CFGs (Penn TreeBank): 20-50 rules, depth 5-8, lexical terminals

→Synthetic CFGs: controlled complexity — **up to 7 levels**, numeric terminals, tunable ambiguity

→Key advantage: **ground-truth parse trees** are always available (we generated the data!)

→Enables precise evaluation of what the model has learned internally

> **Why synthetic?** Real language conflates hierarchical structure with lexical semantics, world knowledge, and pragmatics. Synthetic CFGs isolate *pure structural reasoning*.



Model Variants: Four Architectures Under Test

→All models share **GPT-2-small** base: 12 layers, 12 heads, 768-dim

→The critical variable: **position encoding type**

→DeBERTa (encoder) serves as a bidirectional baseline — it should *not* need global planning

> **Hypothesis:** Position encoding determines whether transformers can learn hierarchical structure. Relative/rotary PE should outperform absolute PE.

GPT-2 (Absolute PE)
12L / 12H / 768d
Vanilla position embeddings
Struggles with deep hierarchy

GPT-2 (Relative PE) ★
12L / 12H / 768d
DeBERTa-style rel. attention
Near-perfect accuracy

GPT-2 (Rotary PE) ★
12L / 12H / 768d
RoPE (Su et al., 2021)
Near-perfect accuracy

DeBERTa (Encoder) ✗
Bidirectional, relative PE
MLM pre-training objective
Cannot learn deep NT info

Key Architectural Differences

Autoregressive (GPT): causal mask, next-token prediction
Encoder (DeBERTa): bidirectional, masked-language modeling

Succeeds at hierarchical CFG learning Fails (pattern matching only)

All models trained on A100 GPUs with synthetic CFG data

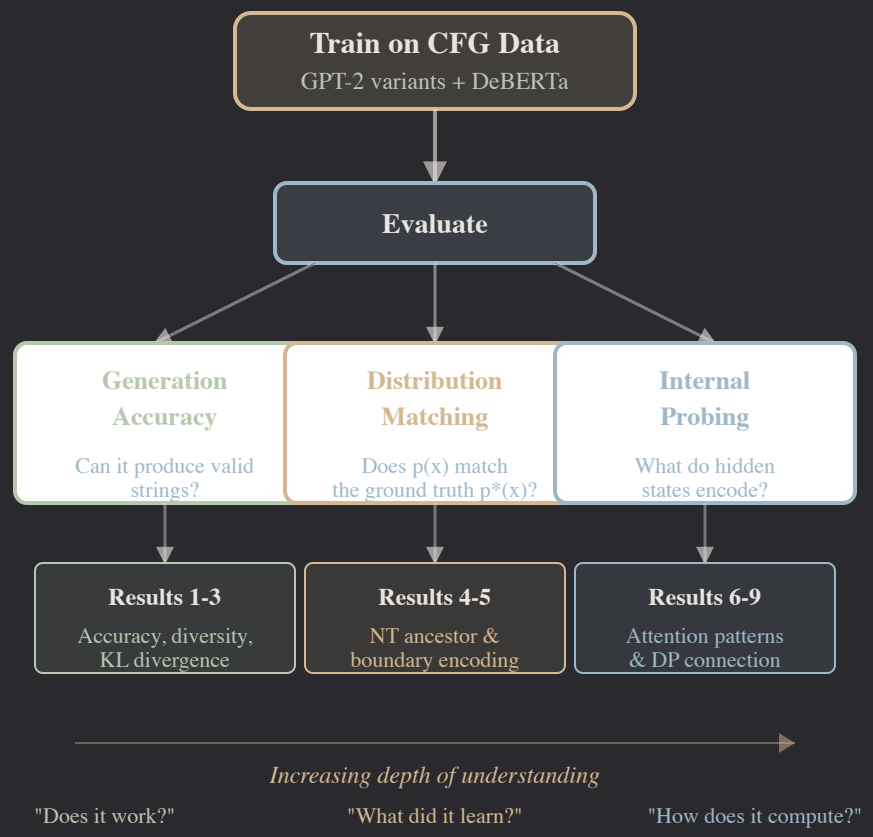
Three-Pronged Evaluation Strategy

→**Prong 1 — Generation accuracy:** Can the model produce valid strings from the CFG? Tests whether it learned the grammar rules.

→**Prong 2 — Distribution matching:** Does the model's probability distribution over strings match the true CFG distribution? (Entropy + KL divergence)

→**Prong 3 — Internal probing:** What do the hidden states encode? Linear probes test whether parse-tree structure is linearly recoverable.

Together, these three evaluations go beyond "does it work?" to answer "what did it learn?" and "how does it compute?"



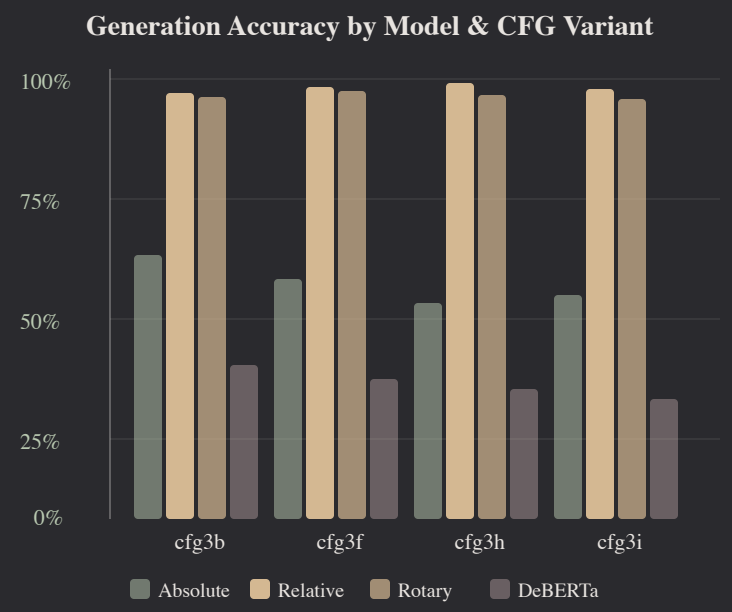
Can Transformers Learn CFGs?

Results 1–3: Generation accuracy, diversity, and distribution matching

Result 1: Near-Perfect Generation Accuracy

Results 1–3

- → GPT-2 with **relative PE** and **rotary PE** achieves near-perfect accuracy on all CFG variants (cfg3b–cfg3i)
- → The model generates grammatically valid strings with **>95% accuracy** even for deep hierarchies
- → GPT-2 with **absolute PE** struggles — especially on deeper structures
- → DeBERTa (encoder) largely **fails** — bidirectional models don't need global planning



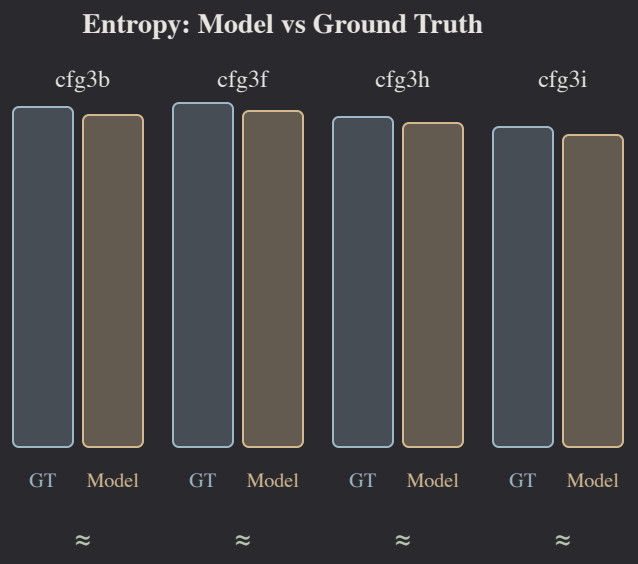
Schematic rendering of Fig. 2 (left). See paper for exact values.

Key Takeaway: Relative and rotary positional encodings enable near-perfect CFG learning. Absolute PE and encoder models fail, suggesting that **relative distance information** is essential for learning hierarchical structure.

Result 2: High Generation Diversity (Birthday Paradox Test)

Result 2: Diversity

- → High accuracy could come from **memorization** — generating the same few valid strings repeatedly
- → The **birthday paradox test** checks: if generating N strings, do we see duplicates earlier than expected?
- → Model entropy closely matches the **ground-truth CFG entropy** across all variants
- → Transformers produce **diverse** strings, sampling from the full distribution — not memorizing a subset



Schematic of Fig. 2 (center). Heights reflect relative entropy.

Implication: The model is not memorizing a lookup table of valid strings. It has learned the **generative process** of the CFG, sampling from a distribution with near-identical entropy to the ground truth.

Result 3: Distribution Matching via KL Divergence

The strongest test: does the model's **next-token probability distribution** match the true CFG distribution?

KL Divergence (averaged over positions):

$$\frac{1}{|S|}\sum_{x\in S} \frac{1}{m(x)+1} \sum_i \sum_t p^*(t|x_{

Where $p^*(t|x_{

GPT-2 with relative/rotary PE achieves **near-zero KL divergence** across all variants — token-level probabilities match the ground truth closely.

Significance: The KL divergence test is the most demanding — it requires matching **every conditional probability**, not just generating valid strings. Near-zero KL means the model has genuinely learned the CFG distribution.

Position Encoding is the Critical Design Choice

→**Relative PE (DeBERTa-style)** and **Rotary PE (RoPE)** succeed across all metrics

→**Absolute PE** struggles: it cannot easily represent *relative distances* between tokens needed for bracket matching

→**DeBERTa (encoder)** fails despite having relative PE — the MLM objective doesn't require global planning

→Key insight: relative distance information is **necessary** for learning hierarchical structure

> **Why does PE type matter?** Hierarchical structure requires knowing the *distance* to the nearest NT boundary. Relative PE naturally encodes this; absolute PE must learn it indirectly.

Position Encoding Impact Summary

Approximate generation accuracy (bars). Exact numbers vary by CFG variant.

Inside the Black Box: What Do Hidden States Encode?

Results 4–5: Linear probing reveals hidden parse-tree structure

Linear Probing: Recovering Parse Structure

→**Key question:** What structural information does the transformer's last-layer representation encode?

→Approach: train a **linear probe** on the hidden states to predict NT ancestor indices and boundary indicators

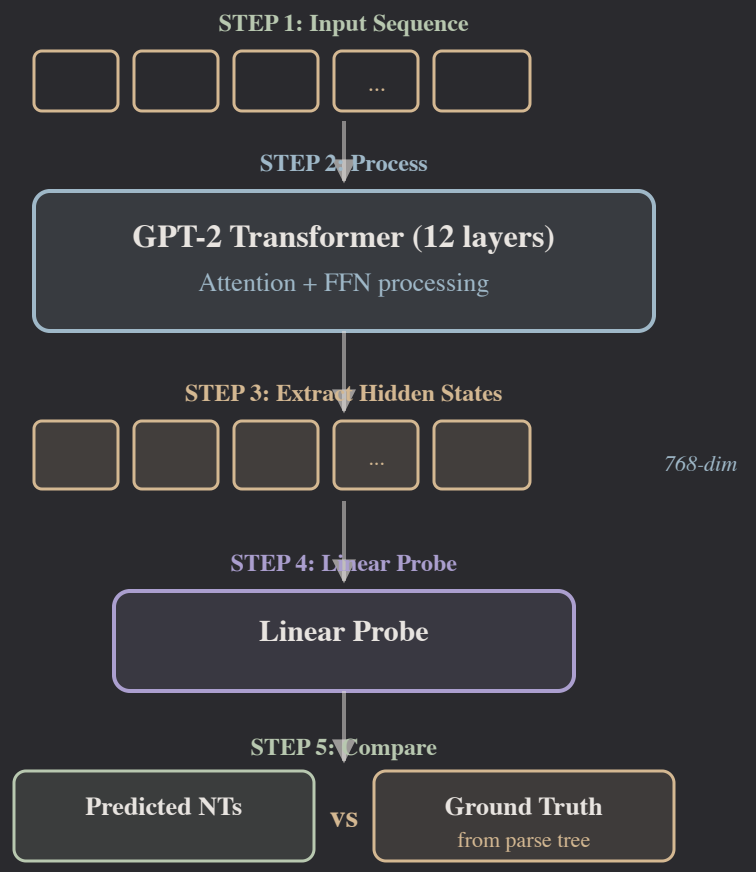
→If a simple linear classifier succeeds, the information is **linearly accessible** in the representation — not buried in nonlinear entanglement

Full-Attention Linear Probe:

$$G_i(x) = \sum_{r \in [H],\, k \in [m(x)]} w_{r,i \to k} \cdot f_r(E_k(x))$$

$E_k(x)$: hidden state at position k | f_r: per-head probing function | $G_i(x) \in \mathbb{R}^{|\mathrm{NT}|}$: predicted NT ancestors

Think of it as an "X-ray" for the transformer: if the probe works, the model has learned the parse tree.



If accuracy is high, the model has learned the parse structure.

Result 4: Hidden States Encode Full NT Ancestor Information

Result 4

→Linear probes on the **last layer** hidden states achieve **near-perfect accuracy** predicting NT ancestor indices at every position

→This holds across all CFG variants for GPT-2 with relative and rotary PE

→GPT-2 with **absolute PE** shows reduced accuracy, especially for deeper NT levels

→**DeBERTa (encoder)** cannot encode deep NT information — MLM only needs local patterns

→Information is concentrated in the **last layer**; middle layers show partial encoding

NT Ancestor Probing Accuracy

By model variant and NT level (cfg3h)

Model					Overall
Relative PE	**99%**	**98%**	**97%**	**96%**	**~98%**
Rotary PE	**98%**	**97%**	**96%**	**94%**	**~96%**
Absolute PE	85%	72%	58%	42%	~64%
DeBERTa	60%	38%	25%	15%	~35%

Increasing hierarchy depth (harder)

High accuracy (>90%) Moderate (60-90%) Low (<60%)

Approximate values from Fig. 3. Exact numbers in paper.

is deepest NT level; is closest to root.

Result 4: The transformer's last-layer hidden states **almost perfectly encode** the NT ancestor and NT boundary information via **linear** probing. Encoder models (DeBERTa) cannot — confirming that autoregressive models build deeper structural representations.

Result 5: NT Information is Local — Concentrated at Boundaries

Result 5

→Result 4 used a *full-attention* probe (accesses all positions). But where is the information **actually stored**?

→Restrict the probe to a **local window** around each position: $|i - k| \leq \delta$

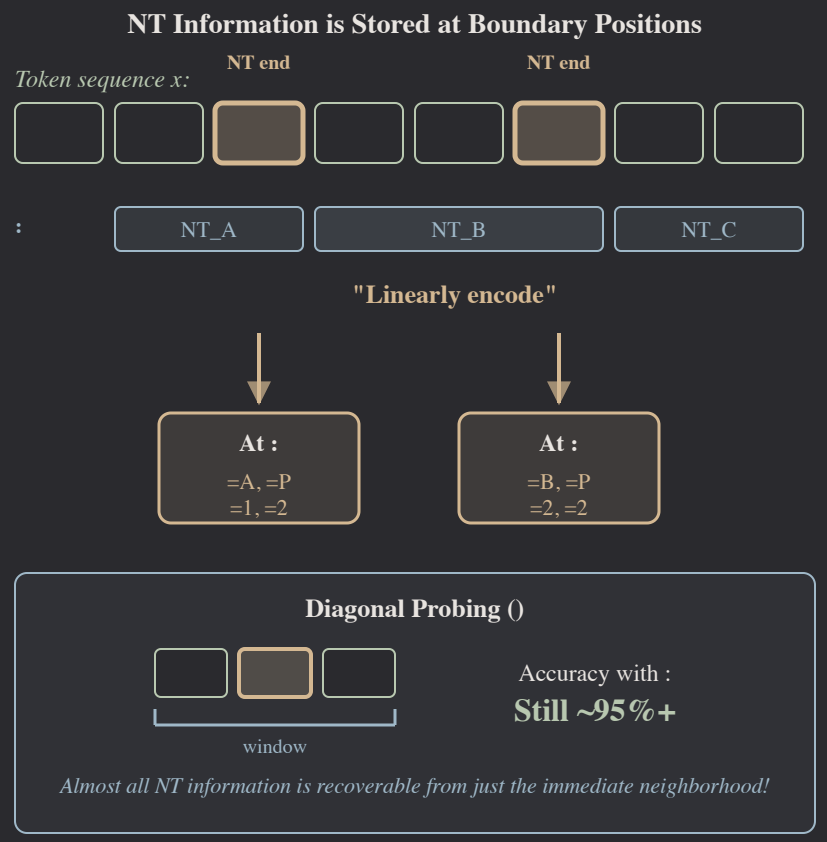
→Even with $\delta = 1$ (diagonal probing — only position $i \pm 1$), accuracy remains high!

Local-window probe: $G_i(x) = \sum_{r,\, |i-k| \leq \delta} w_{r,i \to k} \cdot f_r(E_k(x))$

→NT boundary positions carry the **richest information** — they linearly encode the NT ancestor vectors

→Non-boundary positions carry much less structural info

Intuition: The transformer "writes" the parse tree structure at each NT boundary — like closing brackets that summarize everything inside their span.



Inspired by plots.pdf p.6 (fig:illustrate-nt-at-ntend)

Diagonal Probing Confirms Locality

Result 5 — Evidence

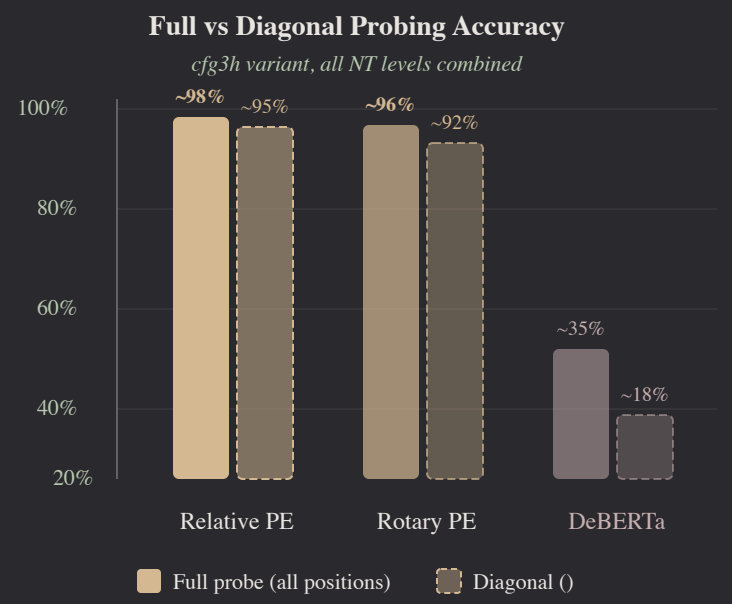
→**Diagonal probing**: restrict the linear probe to only positions $i \pm 1$ (the token itself and its immediate neighbors)

→For GPT-2 with relative/rotary PE: accuracy drops only marginally compared to full-attention probing

→The **gap is larger for encoder models** (DeBERTa), which spread information more globally

→Confirms that GPT-2 encodes NT information **right at the boundary**, not distributed across the sequence

→This is consistent with a **dynamic programming** interpretation: each boundary position accumulates parse state from below



Approximate values from Fig. 5. Small drop for GPT; large drop for DeBERTa.

Conclusion: GPT-2 concentrates parse information **locally at NT boundary positions**. The full-attention probe barely outperforms the diagonal one — meaning almost all structural information is available within a window of ± 1 token around each boundary. This locality is a signature of **dynamic-programming-style computation**.

How Do Transformers Parse? The Attention Mechanism

Results 6–9: Multi-scale attention, NT boundary bias, and the connection to CKY dynamic programming

Result 6: Multi-Scale Position-Based Attention

→Vanilla GPT-2 (absolute PE) develops **multi-scale distance patterns** across its 12 attention heads

→Each head specializes in a **different distance range** — some attend to nearby tokens, others to far-away positions

→Patterns are purely position-based: attention weights depend on positional distance, not on token identity

→This is the **first building block** of the learned parsing mechanism

> **Key Insight:** Multi-scale attention provides the positional scaffolding needed to localize NT boundaries at different hierarchy levels.

Figure: Schematic of multi-scale attention (see paper Fig. 5 for original)

Result 7: Attention Biased Toward NT Boundaries

→Measured the **attention bias** $B_{l,h,j\to i}$ at positions near NT-end tokens across all layers l and heads h

→Tokens at **NT boundary positions** receive disproportionately high attention — far above what position alone would predict

→The bias appears at multiple hierarchy levels ($\ell = 2, 3, 4, 5$) and is strongest at offset $\delta = 0$ (exactly at the boundary)

→This holds across all CFG variants (cfg3b through cfg3i)

> **Implication:** The transformer has learned to *identify* which tokens are NT boundaries and to preferentially attend to them — a prerequisite for boundary-based parsing.

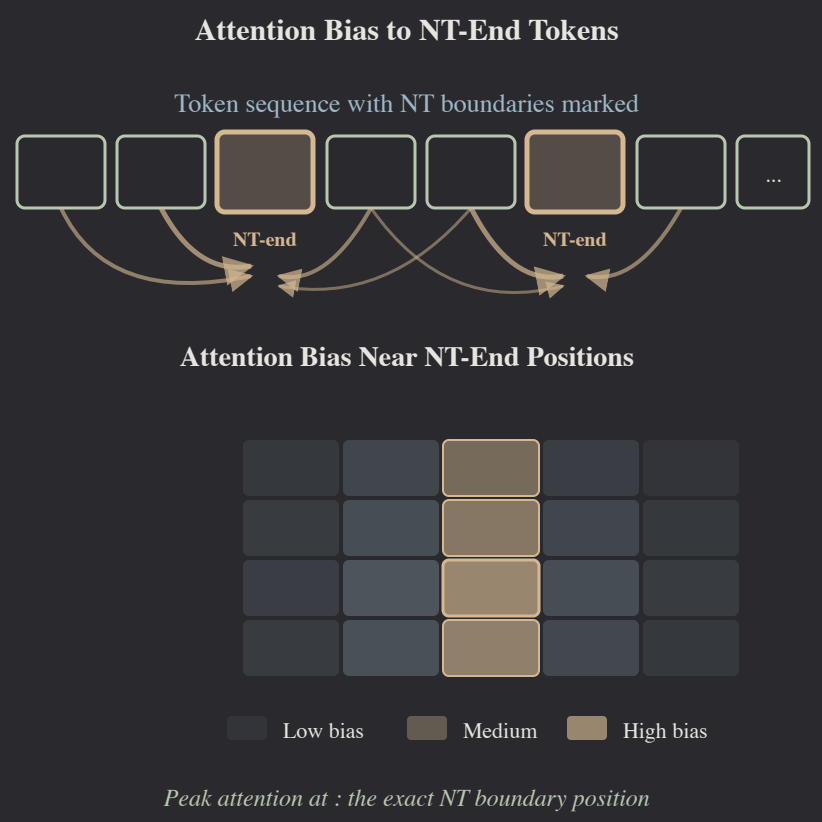


Figure: Attention bias schematic (see paper Fig. 6a for data)

Result 8: NT-End to NT-End Attention at Same Level

→Attention is strongest when **both source and target** tokens are at NT-end positions

→Measured $B_{l,h,j\to i}$ conditioned on *both* positions $i + \delta_1$ and $j + \delta_2$ being at NT boundaries

→The strongest signal: offsets $\delta_1 = \delta_2 = 0$, meaning **exact boundary-to-boundary** attention

→This pattern is **level-specific**: heads at level $\ell = 4$ attend between level-4 boundaries, not between level-3 and level-5

> **Why it matters:** Same-level NT-end attention is exactly what a CKY parser needs — matching opening and closing brackets at each depth of the hierarchy.

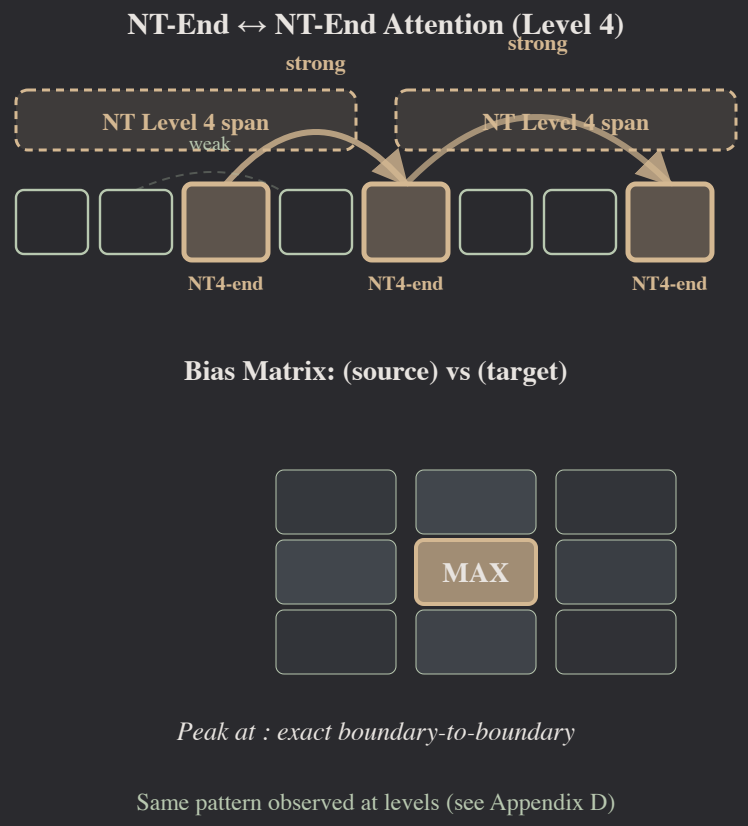


Figure: NT-end attention schematic (see paper Fig. 6b)

Result 9: Adjacent NT-End Attention Patterns

→Attention concentrates between **"adjacent" NT-end token pairs** across hierarchy levels

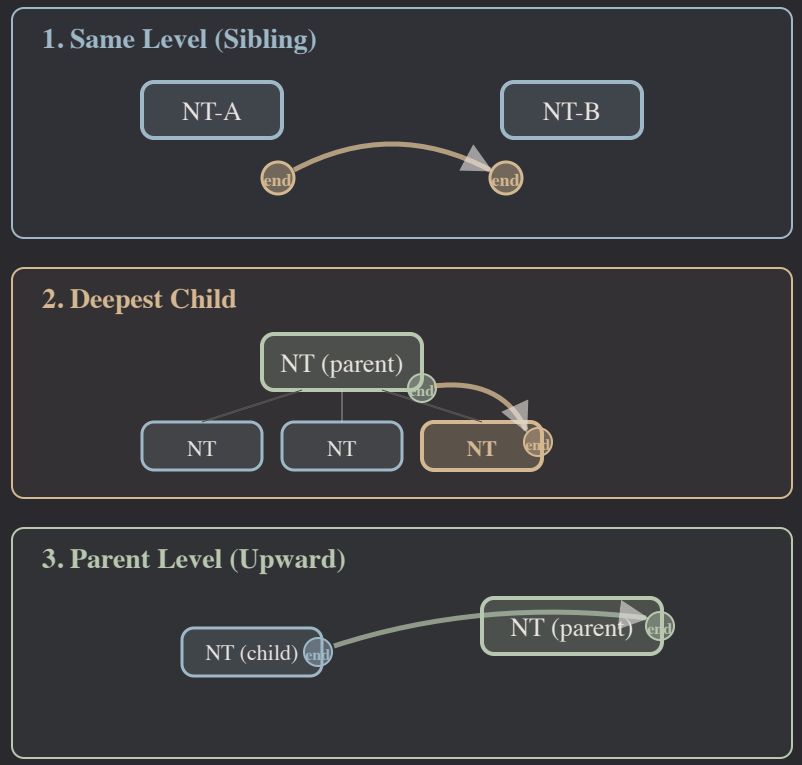
→Three types of adjacency discovered:

- →**Same level** ($\ell' \to \ell$): sibling NT boundaries
- →**Deepest child**: parent boundary attends to its last child's boundary
- →**Parent level**: child boundary attends upward to parent boundary

→Measured as $B^{\text{end}\to\text{end}}_{l,h,\ell'\to\ell,r}$ for level pairs and head ranks r

> **Takeaway:** Results 7–9 collectively show the transformer builds an *implicit bracket-matching system* — not through explicit rules, but through emergent attention patterns.

Three Types of Adjacent NT-End Attention



All three patterns mirror CKY's span-combination steps

Figure: Three adjacency types (see paper Fig. 6c for data)

The Connection to CKY Dynamic Programming

→The CKY algorithm parses a string by building a **DP table** $\mathrm{DP}(i, j, a)$ bottom-up over spans (i, j) with nonterminal a

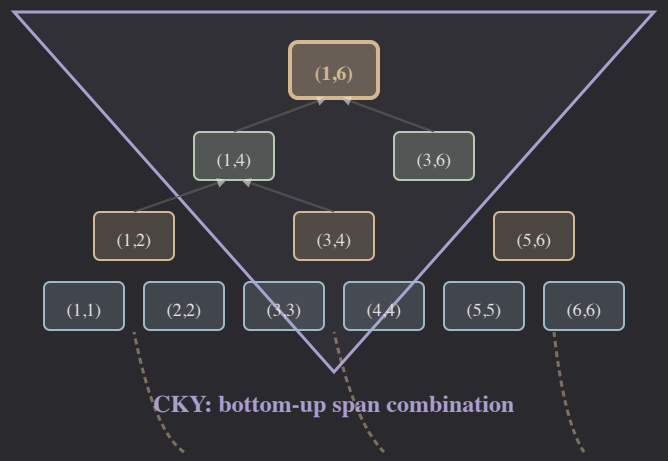
→The transformer's attention patterns **mirror this recurrence**: boundary indicators $\mathbf{1}_\ell$ and ancestor values s_ℓ encode the DP state

→NT boundary detection (Results 4–5) provides the **row/column indices** of the DP table

→Level-specific attention (Results 7–9) provides the **span-combination step**

DP Predicate: If $\mathbf{1}_\ell(i) = 1$, $\mathbf{1}_\ell(j) = 1$, all $\mathbf{1}_\ell(k) = 0$ for $k \in (i, j)$, and $s_\ell(j) = a$, then $\mathrm{DP}(i, j, a) = 1$.

CKY DP Table ↔ Attention



Bold = NT boundary tokens

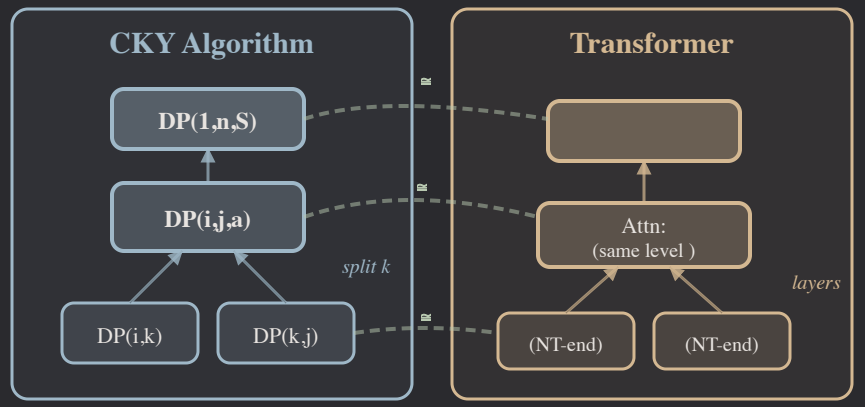
NT boundaries index the DP table

Corollary: GPT *mimics dynamic programming* — the attention mechanism implements a soft, differentiable version of CKY parsing. Boundary-to-boundary attention across layers corresponds to the bottom-up span-combination recurrence of classical chart parsing (Baker, 1979; Sakai, 1961).

CKY vs Transformer: A Side-by-Side Comparison

	Classical CKY	Transformer (learned)
Input	Token sequence + explicit grammar rules	Token sequence only (rules learned implicitly)
State	DP table $\mathrm{DP}(i, j, a)$	Hidden states $E_k(x)$ encoding s_ℓ, $\mathbf{1}_\ell$
Span ID	Explicit indices (i, j)	NT boundary indicators $\mathbf{1}_\ell(i) = 1$
Combine	$\mathrm{DP}(i, k) \oplus \mathrm{DP}(k, j)$ for all split points k	Boundary-to-boundary attention across layers
Direction	Bottom-up over span length	Bottom-up over transformer layers
Output	Parse tree $\mathrm{DP}(1, n, S)$	Next-token probabilities (parse is implicit)

Key Difference: CKY requires *explicit rules* and runs in $O(n^3)$. The transformer learns the rules from data and implements a **soft, parallel approximation** using



Both algorithms:
Identify span boundaries → combine sub-spans bottom-up → produce output

CKY Advantages
- Exact: guaranteed correct parse
- Interpretable table structure
- Works with any given CFG
- Requires explicit grammar
- sequential

Transformer Advantages
- Learns rules from data alone
- Parallelizable across positions
- Generalizes beyond training CFG
- Approximate (soft attention)
- Requires sufficient depth (layers)

Figure: CKY ↔ Transformer structural correspondence (SVG S11)

08

Extensions: Implicit CFGs & Robustness

Do the findings hold beyond clean, explicit context-free grammars? Results 10–13 test generalization to implicit structure and noisy training conditions.

Result 10: Learning Implicit CFGs

→In real language, words don't have a 1-to-1 mapping to grammar roles — many words can fill the same syntactic slot

→**Implicit CFGs**: each non-terminal maps to a *bag of words* rather than a single terminal symbol

→The transformer still learns accurate generation, even with 300 words per NT category

→**Hidden representations** on tokens in the same NT subset show *divergence-correlated* embeddings — a clear internal signature of latent structure

> **Why this matters:** Implicit CFGs are a major step toward real language, where "the" and "a" share a determiner role. The transformer discovers this latent grouping without any labels.

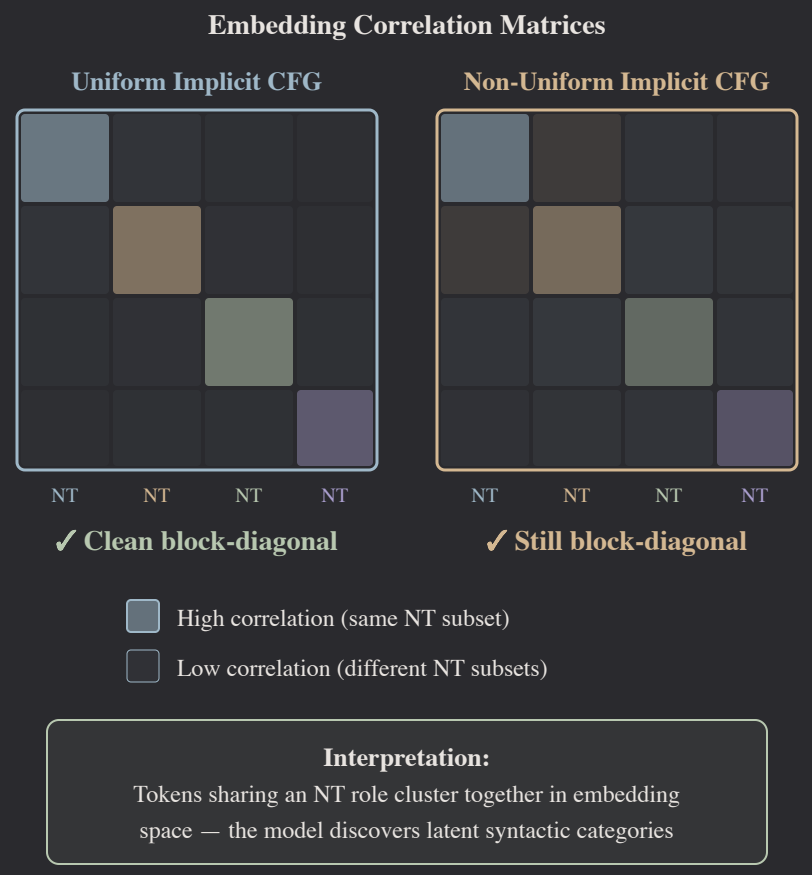


Figure: Embedding correlations for implicit CFGs (paper Fig. 7)

Results 11–13: Robustness to Corrupted Training Data

→ R11 With corrupted training data ($\rho = 15\%$ random tokens), models are **not inherently robust** to noise alone

→ R11 But seeing *both* clean and corrupted data together is **beneficial** — it strengthens generation quality

→ R12 Corrupted tokens cause a dramatic **"mode switch"** in generation behavior (see next slide)

→ R13 Robust generation requires **low temperature** $\tau \leq 0.2$ — encouraging high-probability tokens restores correct structure

Robustness Experiment Design

Training	Test Prefix	Key Finding
Clean only	Clean	✓ Correct
Clean only	Corrupted	✗ Fails
Mixed (85/15)	Clean	✓ Correct
Mixed (85/15)	**Corrupted**	**Mode switch!**
Corrupt only	Clean	**Not robust**

Temperature effect (R13): restores robust generation

Key pattern: Mixed training data (clean + 15% corrupted) is better than either alone. The corruption fraction ρ controls the robustness frontier, and temperature τ acts as a sharpening knob. Consistent across cfg3b, cfg3h, cfg3i variants (Appendix F).

The Mode Switch: Clean vs Corrupted Generation

→When trained on mixed data (85% clean + 15% corrupted), the model develops a **"mode switch"**:

→**Clean prefix** → model generates correctly, following the true CFG distribution

→**Corrupted prefix** → model *always* generates random sentences of **valid length** (not garbage!)

→**Mixed prefix** → model generates length-valid strings with reduced accuracy

> **Analogy:** Like an experienced speaker who can tell whether a conversation is "on track" or "off the rails" and adjusts their output accordingly — not perfectly, but structurally.

Temperature $\tau \leq 0.2$ sharpens the distribution enough to recover correct generation even from mildly corrupted prefixes (Result 13).

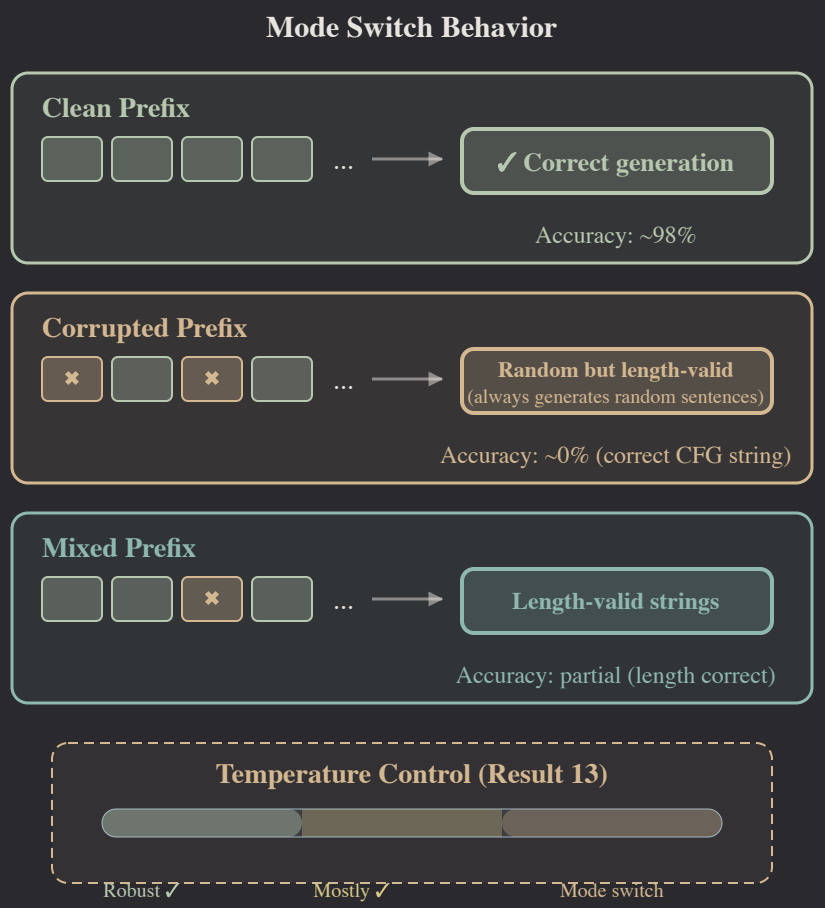


Figure: Mode switch behavior under corruption (SVG S12)

From Synthetic CFGs to Real Language Understanding

→Synthetic CFGs isolate **compositional structure** from lexical semantics — a controlled "physics experiment" for language models

→The discovered mechanisms (boundary-based DP attention) are **architecture-general** — they emerge from standard transformer training, not from custom design

→Implicit CFGs (Result 10) show the bridge: real language has similar latent categories, and the transformer finds them without supervision

Limitations: Synthetic CFGs lack lexical ambiguity, semantic composition, and pragmatic context. Real language hierarchy is deeper and noisier. The 12-layer GPT-2 may not scale the same way as larger models.

Open question: Do frontier LLMs (GPT-4, Claude, Llama) implement similar boundary-based DP mechanisms for real syntactic parsing?

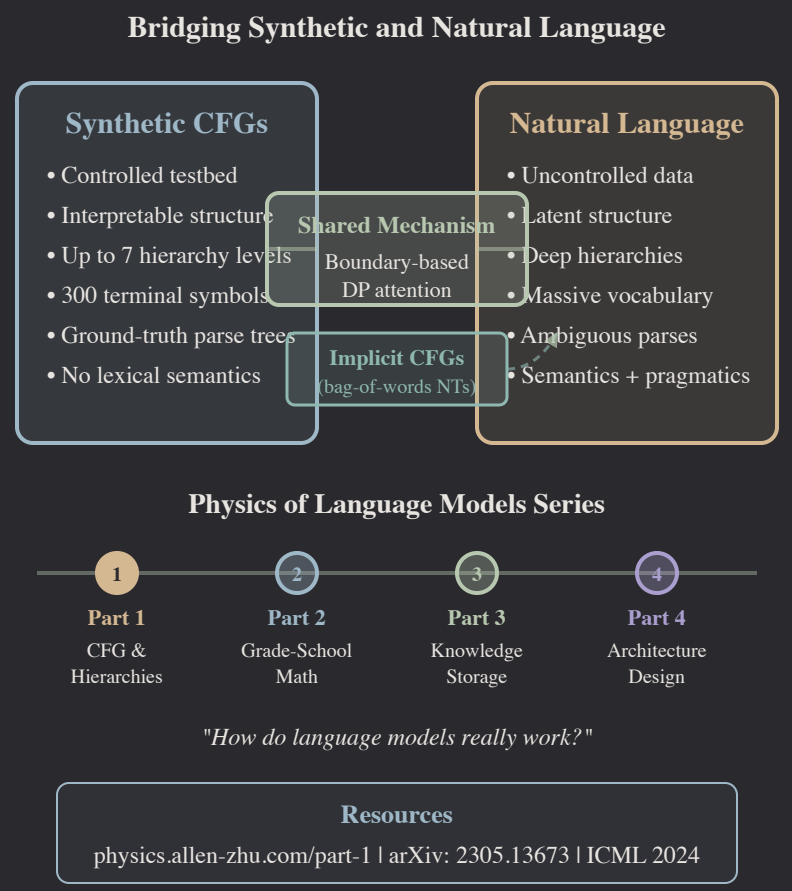


Figure: Synthetic-to-real bridge & Physics of LMs series (SVG S13)

Learning Hierarchical Language Structures

1. **Controlled CFG Testbed** — Multi-level synthetic grammars that isolate hierarchical reasoning from lexical semantics.
   Results 1–3: Accurate learning, high diversity, distribution matching

2. **Hidden Parse Trees** — Linear probes recover full NT ancestor structure from the transformer's last-layer hidden states.
   Results 4–5: Locally encoded at boundary positions

3. **DP-like Attention** — Attention patterns mirror CKY dynamic programming: multi-scale, boundary-biased, level-specific.
   Results 6–9: Mechanistic explanation of how transformers parse

4. **Implicit CFGs** — Latent syntactic categories emerge even when terminals are bags of words.
   Result 10: Divergence-correlated embeddings reveal hidden structure

5. **Robustness & Mode Switch** — Corruption causes a mode switch; low temperature restores structural generation.
   Results 11–13: Practical insights for training under noise

Thank You